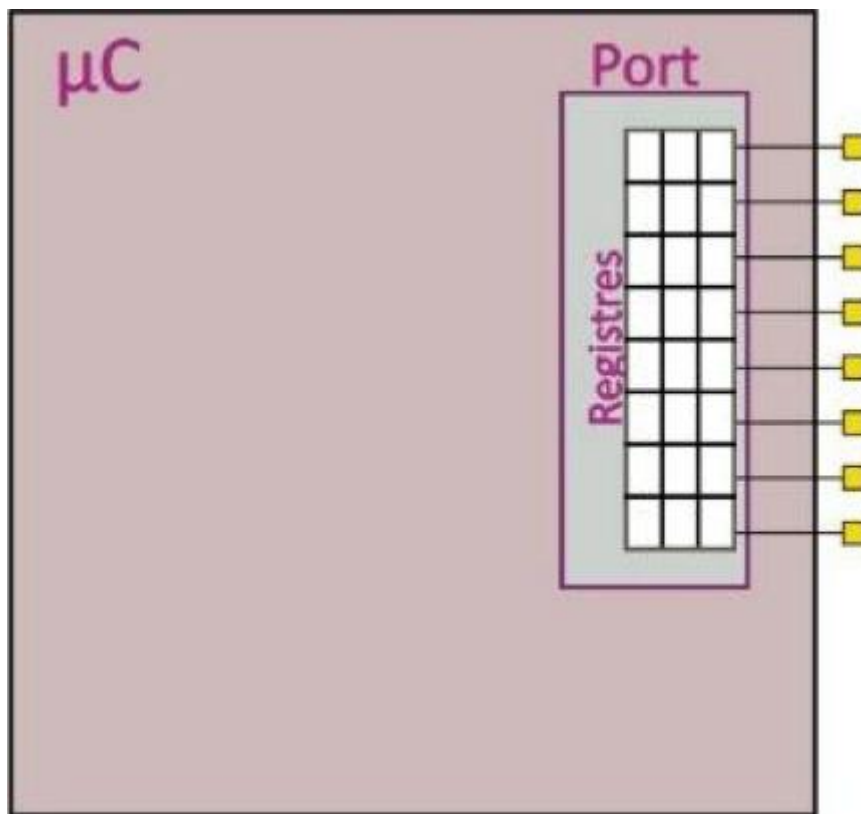


Configurez les ports d'entrée/sortie

Après avoir vu des généralités sur la configuration des périphériques, commençons par nous intéresser à ceux qui offrent le moyen d'interfacer le monde numérique avec le monde physique : les ports d'entrée/sortie. Vous allez découvrir dans la suite de ce chapitre ce que sont ces fameux GPIO, ainsi que les moyens pour les configurer et les contrôler.

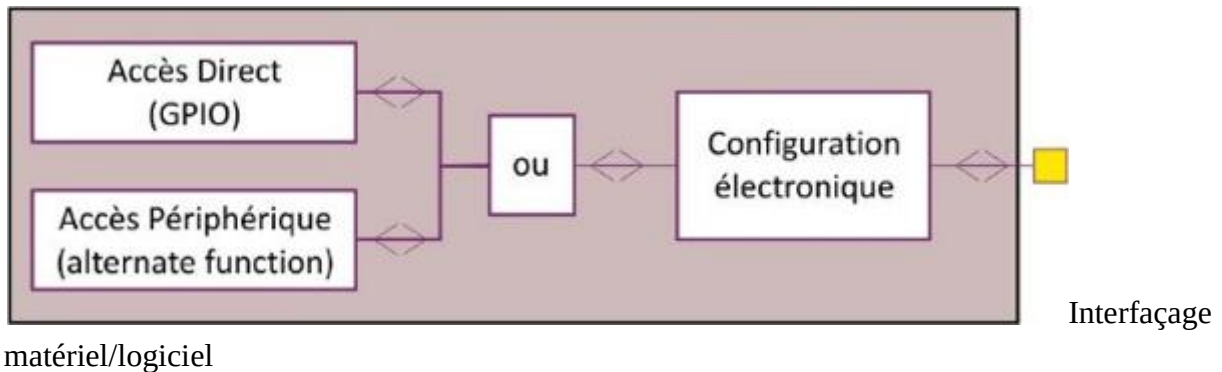
Comprendre les GPIO en général

Un microcontrôleur interagit avec les éléments extérieurs par le biais de plusieurs fils que l'on appelle **broche**, ou *pin* en anglais. Ces broches sont regroupées généralement par paquet de 8 (octet) ou 16 (mot) pour former ce que l'on nommera par la suite des **ports** d'entrée/sortie (*I/O ports*). Du point de vue logiciel, un port est un ensemble de registres.



Un port dans un microcontrôleur

Une broche peut être pilotée soit directement par le biais de registres dédiés (on parle alors de *General Purpose Input Output, GPIO*) soit indirectement par le biais d'un périphérique (on parle alors d'*Alternate Function*). Dans tous les cas, il sera nécessaire de configurer l'étage électronique qui permet d'interfacer le matériel et le logiciel.



matériel/logiciel

Dans la grande majorité des cas, les différentes broches d'un même port sont configurables indépendamment des autres broches.

Une broche d'un port peut être configurée en entrée ou en sortie (le sens étant celui vu du microcontrôleur), c'est ce que l'on appelle sa **direction**.

Lorsqu'une **broche est mise en entrée** il est possible d'observer des signaux entrants dans le microcontrôleur. En fonction de sa configuration, nous pouvons directement exploiter la valeur physique du signal en la convertissant en une valeur numérique (nous verrons cela plus en détail dans la partie dédiée à l'ADC), ou bien l'observer sous une forme binaire logique (0 ou 1), les états haut et bas étant fixés en fonction d'un seuil sur la tension en entrée. Dans le dernier cas d'accès direct (GPIO), un registre appartenant au port permet au logiciel, par une simple lecture, de connaître l'état logique de chacune des broches composant le port.

Lorsqu'une broche **est configurée en sortie**, il est possible de fixer une tension sur une broche. Dans le cas de l'utilisation des *alternate functions* cette tension est fixée par un autre périphérique (UART, convertisseur numérique-analogique, PWM, etc.). Dans le cas d'accès direct, un registre propre au port impose une tension haute (valeur logique 1) ou basse (valeur logique 0) au circuit qui lui est connecté.

Sur la majorité des microcontrôleurs actuels les broches peuvent être configurées soit en entrée soit en sortie. Pour utiliser l'accès direct un port possède donc au moins un registre de configuration qui spécifie pour chaque broche sa direction, un registre pour lire l'état binaire des broches mises en entrée et un registre pour imposer l'état binaire des broches mise en sortie.

Et sur le STM32F103 comment fait-on ?

Sur cette famille de microcontrôleur, le nombre de ports disponibles varie en fonction de la taille du boîtier. Pour la carte nucleo qui vous a été proposée, le boîtier LQFP64 a seulement quatre ports de disponibles. Ils sont identifiés par les lettres A, B, C et D. Les ports A, B et C ont chacun 16 broches et le port D seulement 3. Le STM32F103-LQFP64 peut avoir au mieux 51 broches utilisables pour s'interfacer avec un process. A noter que pour d'autres versions de ce processeurs (en boîtier LFPQ100 par exemple), il est possible de trouver jusque 7 ports GPIO, d'où l'existence dans la documentation des ports E, F et G qui n'ont pas de réalité dans le microcontrôleur sur lequel nous basons nos exemples.

Pour désigner un port, nous utiliserons par la suite la notation GPIO suivie de la lettre le désignant. Ainsi GPIOA désigne le port A. On ajoutera parfois un point suivi d'un nombre pour désigner la broche, ainsi GPIOB.10 désigne la broche 10 du port B. Comme les différents ports sont structurés et fonctionnent de la même manière, la documentation fait généralement référence aux GPIOx, où x est à remplacer par la lettre correspondante au port concerné.

Trouver la bonne adresse

Comme nous l'avons dit dans le chapitre précédent, différents registres permettent de configurer un périphérique. Ces registres ont des adresses mémoires spécifiques qu'il faut connaître si nous voulons fixer une configuration. Il faut donc commencer par trouver ces adresses. Pour cela, vous n'avez pas le choix, il faut rentrer dans la documentation technique du microcontrôleur !

Pour la famille des microcontrôleurs STM32F1xx, le document de référence est le [RM0008](#) disponible sur le site de la société ST. Nous pouvons y trouver la liste des adresses associées à chaque périphérique. Ainsi, nous découvrons page 50 que, par exemple, l'ensemble des registres du port B sont fixés sur la plage mémoire de 0x4001 0C00 à 0x4001 0FFF.

Table 3. Register boundary addresses (continued)

Boundary address	Peripheral	Bus	Register map
0x4001 5800 - 0x4001 7FFF	Reserved	APB2	-
0x4001 5400 - 0x4001 57FF	TIM11 timer		Section 16.5.11 on page 469
0x4001 5000 - 0x4001 53FF	TIM10 timer		Section 16.5.11 on page 469
0x4001 4C00 - 0x4001 4FFF	TIM9 timer		Section 16.4.13 on page 459
0x4001 4000 - 0x4001 4BFF	Reserved		-
0x4001 3C00 - 0x4001 3FFF	ADC3		Section 11.12.15 on page 252
0x4001 3800 - 0x4001 3BFF	USART1		Section 27.6.8 on page 832
0x4001 3400 - 0x4001 37FF	TIM8 timer		Section 14.4.21 on page 362
0x4001 3000 - 0x4001 33FF	SP11		Section 25.5 on page 745
0x4001 2C00 - 0x4001 2FFF	TIM1 timer		Section 14.4.21 on page 362
0x4001 2800 - 0x4001 2BFF	ADC2		Section 11.12.15 on page 252
0x4001 2400 - 0x4001 27FF	ADC1		Section 9.5 on page 193
0x4001 2000 - 0x4001 23FF	GPIO Port G		
0x4001 1C00 - 0x4001 1FFF	GPIO Port F		
0x4001 1800 - 0x4001 1BFF	GPIO Port E		
0x4001 1400 - 0x4001 17FF	GPIO Port D		
0x4001 1000 - 0x4001 13FF	GPIO Port C		
0x4001 0C00 - 0x4001 0FFF	GPIO Port B		
0x4001 0800 - 0x4001 0BFF	GPIO Port A		
0x4001 0400 - 0x4001 07FF	EXTI		Section 10.3.7 on page 213
0x4001 0000 - 0x4001 03FF	AFIO		Section 9.5 on page 193

Registres du port B

Mais comment trouver les différents registres d'un port ?

Pour répondre à cette question, il faut de nouveau explorer la documentation — vous aurez compris que sans consulter les documents techniques, il est impossible de faire le travail — et se référer au chapitre dédié au GPIO (chapitre 9, page 158). Au début du chapitre, nous trouvons une description générique du périphérique et à la fin un tableau qui présente sous forme synthétique l'ensemble de ses registres (p. 193). Une description bit à bit des registres est aussi faite dans des sections dédiées.

Table 59. GPIO register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOx_CRL	CN F7 [1:0]	MODE 7 [1:0]		CN F6 [1:0]	MOD E6 [1:0]		CN F5 [1:0]	MOD E5 [1:0]		CNF 4 [1:0]	MOD E4 [1:0]		CN F3 [1:0]	MODE 3 [1:0]		CNF 2 [1:0]	MODE 2 [1:0]		CN F1 [1:0]	MOD E1 [1:0]		CNF 0 [1:0]	MOD E0 [1:0]									
	Reset value	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0
0x04	GPIOx_CRH	CN F15 [1:0]	MODE 15 [1:0]		CN F14 [1:0]	MOD E14 [1:0]		CN F13 [1:0]	MOD E13 [1:0]		CNF 12 [1:0]	MOD E12 [1:0]		CN F11 [1:0]	MODE 11 [1:0]		CNF 10 [1:0]	MODE 10 [1:0]		CN F9 [1:0]	MOD E9 [1:0]		CNF 8 [1:0]	MOD E8 [1:0]									
	Reset value	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0
0x08	GPIOx_IDR	Reserved																IDRy															
	Reset value																	000000000000000000000000															
0x0C	GPIOx_ODR	Reserved																ODRy															
	Reset value																	000000000000000000000000															
0x10	GPIOx_BSR R	BSR[15:0]																BSR[15:0]															
	Reset value	000																															
0x14	GPIOx_BRR	Reserved																BR[15:0]															
	Reset value																	00000000000000000000000000000000															
0x18	GPIOx_LCK R	Reserved																LOCK	LCK[15:0]														
	Reset value																		00000000000000000000000000000000														

Documentation GPIO

Le registre contenant l'état des broches est nommé GPIOx_IDR. D'après le tableau, il a un offset de 0x08, ce qui signifie que l'adresse de GPIOx_IDR est celle de base du port x auquel s'ajoute un décalage de 0x08, soit pour le port B une valeur de 0x4001 0C08.

Pour lire la valeur du registre IDR du port B, il est donc possible en C de faire

```
valueIDR = *(int *)0x40010C08; // lecture de la valeur du registre IDR
```

À noter que la variable valueIDR contiendra alors la valeur logique des broches du port B. Pour résumer, si on souhaite manipuler le registre GPIOx_IDR, il faut chercher l'adresse de base du périphérique, puis décaler cette adresse de la valeur d'offset propre au registre.

Vous aurez compris que tout cela peut devenir très vite fastidieux !

Utiliser le travail des autres

Heureusement, ST-MicroElectronics a compris qu'il était nécessaire de faciliter la vie des programmeurs. C'est pourquoi, ils fournissent des fichiers de configuration avec la majorité des microcontrôleurs. Dans notre cas, la société ST a produit le fichier *STM32f10x.h* qui contient un ensemble de définitions qui va grandement nous aider.

Dans ce fichier nous allons trouver des structures C prédéfinies pour accéder plus rapidement aux registres, ainsi qu'un ensemble de variables ayant pour valeur les adresses des périphériques.

Pour les ports, nous trouvons dans le fichier *STM32f10x.h* une structure `GPIO_TypeDef` définie comme

```
Typedef struct
{
    __IO uint32_t CRL;

    __IO uint32_t CRH;

    __IO uint32_t IDR;

    __IO uint32_t ODR;

    __IO uint32_t BSRR;

    __IO uint32_t BRR;

    __IO uint32_t LCKR;
} GPIO_TypeDef;
```

où `__IO uint32_t` est un type pour des valeurs volatiles signées en 32 bits et nous y retrouvons la liste de tous les registres d'un port ordonnés en fonction de leur offset.

Recherchons maintenant dans *STM32f10x.h* toutes les informations relatives au port B. Nous découvrons une adresse pour `GPIOB` définie par

```
#define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
```

et si nous continuons à explorer le fichier de définition, nous trouvons :

```
#define GPIOB_BASE (APB2PERIPH_BASE + 0x0C00)

...

#define APB2PERIPH_BASE (PERIPH_BASE + 0x10000)

...
```

```
#define PERIPH_BASE ((uint32_t)0x40000000)
```

Donc en remontant la chaîne, nous avons GPIOB qui prend la valeur 0x40000000+0x10000+0x0C00 soit 0x4001 0C00 qui est bien l'adresse de base du GPIOB fournie par la documentation.

Pour avoir accès au registre qui vous intéresse, il ne reste plus qu'à utiliser les champs de la structure GPIO_TypeDef. Ainsi, pour lire le registre IDR du port B, nous aurons simplement à écrire :

```
value = GPIOB->IDR ; // value prend la valeur stockée à l'adresse 0x40010C08
```

Configurer la direction

Vous savez comment trouver et manipuler l'adresse d'un registre pour le STM32, maintenant il faut comprendre leur contenu.

Concentrons-nous sur les registres pour configurer les broches d'un port. D'après la documentation, ce sont les registre GPIOx_CRL et GPIOx_CRH qu'il faut utiliser pour configurer la direction d'un port.

8.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, **CNFy[1:0]**: Port x configuration bits (y= 0 .. 7)

23:22, 19:18, 15:14, These bits are written by software to configure the corresponding I/O port.

11:10, 7:6, 3:2

Refer to [Table 17: Port bit configuration table on page 140](#).

In input mode (MODE[1:0]=00):

- 00: Analog input mode
- 01: Floating input (reset state)
- 10: Input with pull-up / pull-down
- 11: Reserved

In output mode (MODE[1:0] > 00):

- 00: General purpose output push-pull
- 01: General purpose output Open-drain
- 10: Alternate function output Push-pull
- 11: Alternate function output Open-drain

Bits 29:28, 25:24, **MODEy[1:0]**: Port x mode bits (y= 0 .. 7)

21:20, 17:16, 13:12, These bits are written by software to configure the corresponding I/O port.

9:8, 5:4, 1:0

Refer to [Table 17: Port bit configuration table on page 140](#).

00: Input mode (reset state)

01: Output mode, max speed 10 MHz.

10: Output mode, max speed 2 MHz.

11: Output mode, max speed 50 MHz.

Documentation : configurer la direction d'un port

Malheureusement, on constate qu'il faut quatre bits pour configurer un seul port et qu'il y a huit configurations possibles. Le tableau ci-après décrit toutes les possibilités.

direction	mode	Description	Valeur du champ de bits
input	Analog	mode d'entrée permettant une acquisition directe de la tension (l'état binaire de la broche n'est plus connue sur le registre)	0000
input	Floating input	mode de base pour une entrée, si aucune tension n'est appliquée, la valeur n'est pas connue	0100

direction	mode	Description	Valeur du champ de bits
input	With pull-up / pull-down	mode d'entrée pour lequel la valeur il est possible de fixer l'état par défaut à 0 (pull down) ou à 1 (pull-up)	1000
input	Analog	Non utilisé	1100
Output(*)	Push-pull	mode de sortie dans lequel la tension appliquée à la broche est toujours forcée à 0 ou 1	0001 0010 0011
Output(*)	Open-drain	mode de sortie dans lequel le niveau de tension haut est fixé par le circuit connecté à la broche	0101 0110 0111
Output(*)	Alternate function push-pull	mode de sortie identique au mode push-pull, mais l'état de la broche est contrôlé par un autre périphérique que le port	1001 1010 1011
Output(*)	Alternate function open-drain	mode de sortie identique au mode open-drain, mais l'état de la broche est contrôlé par un autre périphérique que le port	1101 1110 1111

(*) trois valeurs possible pour définir le port en sortie (01,10 ou 11) qui correspondent à des fréquences d'entrée maximale admissible différentes. La valeur 01 convient très bien pour la grande majorité des cas.

Reprenons : il y a 16 broches sur un port, il faut 4 bits par broche pour la configurer en entrée ou sortie, soit 64 bits pour configurer l'ensemble des broches d'un port. Or un registre fait 32 bits. Il faut donc deux registres pour configurer l'ensemble des broches du même port d'où l'existence de deux registres : CRL et CRH.

Le registre CRL (L pour *low*) permet de configurer les broches de 0 à 7 et le registre CRH (H pour *high*) de 8 à 15.

Supposons que nous voulions configurer la broche 5 du port A en entrée (*input floating*). Il faut alors affecter la valeur binaire 0b0100 aux bits b23 b22 b21 b20 du registre CRL. Pour cela on écrit simplement

```
GPIOA->CRL = GPIOA->CRL & ~(0xF << 20); // Mise à 0 des bits b23 b22 b21 b20
```

```
GPIOA->CRL = GPIOA->CRL | (0x1 << 22); // Mise à 1 du bit b22
```

Dans la même idée, pour mettre la broche 10 du port A en sortie (*output push-pull*), il faut fixer la valeur 0b0001 aux bits b11 b10 b9 b8 dans le registre CRH, soit

```
GPIOA->CRH = GPIOA->CRH & ~(0xF << 8); // Mise à 0 des bits b11 b10 b9 b8
```

```
GPIOA->CRH = GPIOA->CRH | (0 << 8); // Mise à 1 du bit b8
```

Lire les valeurs en entrée

En continuant l'exploration des registres liés à un port, on rencontre le registre IDR qui contient sur ses 16 premiers bits une image de l'état des broches. Pour connaître cette valeur, il suffit donc d'aller lire le registre IDR.

Supposons que nous voulons connaître la valeur de la broche 7 du port C, nous écrivons alors simplement :

```
valueb7 = ( GPIOC->IDR & (0x1 << 7) ) >> 7;
```

Rappelons que si on utilise l'état de la broche pour faire un test, le dernier décalage à droite n'est pas nécessaire et que l'on peut simplement écrire

```
if (GPIOC->IDR & (0x1 << 7)) {  
  
    ...  
  
}
```

condition qui sera vraie si le bit 7 de IDR est à 1 et fausse si le bit est à 0.

Écrire sur un port en sortie

Contrôler l'état d'une broche en sortie n'est pas compliqué. Pour cela nous utilisons le registre ODR, dont les seize premiers bits fixent l'état de chaque broche en sortie d'un port. Ainsi pour mettre à 1 la broche 8 du port B, nous fixons simplement à 1 le bit 8 de ODR, soit

```
GPIOB->ODR = GPIOB->ODR | (1 << 8);
```

On remarquera qu'étant donné que le STM32 n'a pas de mémoire « bit accessible » l'affectation d'un bit nécessite la lecture d'un registre, le masquage adéquat de la valeur ainsi lue et enfin l'affectation du résultat sur le registre ; soit au moins 3 opérations.

Il existe cependant dans l'architecture du Cortex une alternative qui permet d'éviter cela (le « bit-banding »). Sans entrer dans le détail l'idée est de mettre en correspondance un bit avec un mot. STMicro a utilisé ce principe pour créer des registres de SET et de RESET des GPIO. Ce sont les registres BRR (Bit Reset Register) et BSRR (Bit Set and Reset Register). Chaque bit de ces registres correspond à un bit des GPIO et seul le niveau logique 1 a un effet. Le programmeur peut donc directement affecter le valeur 1 en mettant à 1 le bit correspondant du registre BSRR (action SET) et affecter le valeur 0 en mettant 1 le bit

correspondant du registre BRR (action RESET : on agit en mettant 1 pour forcer la mise à 0, ce qui peut être déroutant en première lecture).

Un exemple simple

Passons à un exemple concret. Vous allez créer un programme qui allume la led branchée sur la broche 5 du port A (led verte sur la carte nucleo) et l'éteindre quand le bouton USER est appuyé, bouton qui est branché sur la broche 13 du port C.

Pour trouver sur quelle broche sont connectés les composants de la carte nucleo, il vous faut consulter la document propre à [la carte que l'on trouve en ligne](#). Dans ce document, vous trouverez page 22 que le bouton USER est connecté à la broche PC.13 et que pour la led la broche dépend de la carte. Ici pour la STM32F103 la broche est la PA.5.

Pour réaliser la suite, il vous faut un projet configuré (voir la partie 1) avec un main qui ne fait rien, mais qui inclut le fameux fichier *STM32f10x.h*. Votre fichier *main.c* contient donc uniquement les lignes suivantes :

```
#include "STM32f10x.h"

int main (void)
{

    while(1)

    {

    }

    return 0

}
```

Pour commencer, ajouter dans le main avant la boucle while(1) la ligne

```
RCC->APB2ENR = RCC->APB2ENR | RCC_APB2ENR_IOPAEN |  
RCC_APB2ENR_IOPCEN;
```

sans chercher à comprendre ce que cela fait (nous verrons cela dans le chapitre suivant).

Nous voulons que GPIOA.5 soit en sortie (output push-pull) et GPIOC.13 en entrée (input floating), soit :

- Affecter la valeur b0001 aux bits 20 21 22 23 du registre CRL de GPIOA :

- `GPIOA->CRL = GPIOA->CRL & ~(0xF << 20); // output push-pull b0001`
- `GPIOA->CRL = GPIOA->CRL | (0x01 << 20); // output push-pull b0001`
- Affecter la valeur b0100 = 0x04 au bits 20 21 22 23 du registre CRH de GPIOC :

- `GPIOC->CRH = GPIOC->CRH & ~(0xF << 20); // input floating b0100`
- `GPIOC->CRH = GPIOC->CRH | (0x04 << 20); // input floating b0100`

Pour vérifier que cela est bien fait, compilez le code en simulation, puis lancer le debug. Mettez un point d'arrêt au niveau de la bouclewhile. Ouvrez les fenêtres peripheral->GPIOA et peripheral->GPIOC. Lancez l'application et parcourez les broches pour observer leur configuration.

Maintenant, vous allez mettre en place dans la boucle une scrutation de l'état du bouton USER. Ce bouton est branché sur le GPIOC.13. En écrivant

```
while(1) {

    if (GPIOC->IDR & (0x01 << 13)){

    }

}
```

le bit 13 du registre IDR du port C est évalué à chaque fois que la boucle est exécutée. On regarde donc si le bouton est pressé ou non.

Changeons maintenant l'état de la broche 5 du port A pour allumer et éteindre la led à chaque appui sur le bouton. Pour cela nous allons tester la valeur actuelle de la broche et l'inverser. Ajoutez dans la boucle while(1) :

```
while(1){

    if (GPIOC->IDR & (1 << 13)){

        if (GPIOA->ODR & (1 << 5) { // la broche 10 est dans l'état 1

            GPIOA->ODR = GPIOA->ODR & ~(1 << 5); // On passe à 0 le bit 10 de ODR

        } else {

            GPIOA->ODR = GPIOA->ODR | (1 << 5); // On passe à 1 le bit 10 de ODR

        }

    }

}
```

```

    }

}
}

```

Cette opération peut être réalisée bien plus efficacement avec un XOR avec 1 sur le bit 5, soit en remplaçant le code précédent par les lignes :

```

while(1) {

    if (GPIOC->IDR & (1 << 13)){

        GPIOA->ODR = GPIOA->ODR ^ (1 << 5);

    }

}

```

Pour tester cela, compilez en simulation, placez un point d'arrêt au niveau du test (`GPIOA->ODR = GPIOA->ODR ^ (1 << 5)`) et ouvrez les fenêtres de debug de GPIOA et GPIOC.

Après avoir lancé l'exécution vous pouvez changer l'état du bit 13 de IDR pour le GPIOC en cliquant simplement dans le carré représentant le bit qui vous intéresse (ligne *Pins* dans la fenêtre d'un port).

Faites avancer l'exécution et observez la valeur de ODR de GPIOA. Changez la valeur de IDR pour le GPIOC et observez de nouveau ODR de GPIOA, faites avancer plusieurs fois l'exécution.

Normalement vous constaterez que nous n'avons pas le comportement souhaité car la led change d'état tant que le bouton USER est pressé, or nous voulons seulement modifier la led quand le bouton est pressé.

Une solution simple est donc de simplement mémoriser l'état précédent du bouton en ajoutant une nouvelle variable qui stocke l'état précédent du bouton. Pour cela, déclarez dans le main une variable `state` :

```

void main(void){

    ...

    int state = 0;

    while(1){

```

```
...  
  
}  
  
}
```

Il faut maintenant changer l'état de la led uniquement si state est différent de l'état actuel du bouton et enfin, ne pas oublier de mettre à jour state, soit

```
while(1) {  
  
    if (state != GPIOC->IDR & (1 << 13)){  
  
        GPIOA->ODR ^= (1 << 5);  
  
    }  
  
    state = GPIOC->IDR & (1 << 13);  
  
}
```

Compilez, lancez le débogueur et observez le comportement de votre led quand vous changez l'état de GPIOC.13.

Bravo, vous venez de faire l'équivalent du Hello World matériel !